

Unit 3.3 Process Management

- Process Concepts (studied earlier)
- Threads (studied earlier)
- Inter Process Communication (contd..)

Types of Mutual Exclusion

Mutual exclusion is a property of concurrency control, it is used for the purpose of preventing race conditions.

Different types of mutual exclusion are:

- 1.Semaphore
- 2.Monitors
- 3.Mutexes
- 4.Message Passing
- 5.Bounded Buffer

Semaphore

- In 1965, Dijkstra proposed a new and very significant technique for managing concurrent processes by using the value of a simple integer variable to synchronize the progress of interacting processes.
- A semaphore is a variable used to control access to a common resource by multiple processes in a concurrent system such as a multitasking OS.
- This variable is used to solve critical section problems and to achieve process synchronization in the multiprocessing environment.
- A semaphore (S) is an integer variable that can be accessed only through two standard operations:
 - wait()
 - signal()

Semaphore (contd..)

- The wait() operation reduced the value of semaphore by 1 and the signal() operation increases the value by 1.

```
wait(S)
{
    while(S<=0)  //busy waiting
        S--;
}
Signal(S)
{
    S++;
}
```

Semaphore (contd..)

Types of Semaphore

- Binary Semaphore
- Counting Semaphore

Binary Semaphore

- It is a special form of semaphore used for implementing mutual exclusion. It is often called a Mutex.
- It is similar to counting semaphores, but their value is restricted to 0 and 1.
- In this semaphore, the wait() operation works only if semaphore=1, and the signal() operation succeeds when semaphore=0.
- It is easy to implement than counting semaphores.

Semaphore (contd..)

Counting Semaphores

- It uses a count that helps task to be acquired or released numerous times (process that needs to be CR for multiple times).
- If the initial count=0, the counting semaphore should be created in the unavailable state.
- However, if the count>0, the semaphore is created in available state, and the number of tokens it has equals to its count.

Semaphore (contd..)

Problem Statement: We have a buffer of fixed size. A producer can produce an item and place in the buffer. A consumer can pick items and can consume them. We need to ensure that when a producer is placing an item in the buffer, then at the same time consumer should not consume any item. In this problem, buffer is the critical section.

To solve this problem, we need two counting semaphores – Full and Empty. Full keeps track of number of items in the buffer at any given time and Empty keeps track of number of unoccupied slots.

Initialization of semaphores:

Mutex=1

Full=0 //Initially, all slots are empty

Empty=n //Initially, all slots are empty

Semaphore (contd..)

Solution for producer

```
do
{
    //produce an item
    wait(empty);
    wait(mutex);
    //place in buffer
    signal(mutex);
    signal(full);
}while(true);
```

When producer produces an item then the value of empty is reduced by 1 because one slot will be filled now.

The value of mutex is also reduced to prevent consumer to access the buffer.

Now, the producer has placed the item and thus the value of full is increased by 1.

The value of mutex is also increased by 1 because the task of producer has been accomplished and consumer can access the buffer

Semaphore (contd..)

Solution for Consumer

```
do
{
    wait(full);
    wait(mutex);
    //remove item from buffer
    signal(mutex);
    signal(empty);
    //consumes item
}while(true);
```

As the consumer is removing an item from buffer, therefore the value of full is reduced by 1 and the value of mutex is also reduced. So, the producer cannot access the buffer at this moment.

Now, the consumer has consumed the item, thus increasing the value of empty by 1. The value of mutex is also increased so that producer can access the buffer now.

Properties of Semaphores

- It is simple and always has a non-negative integer value.
- It is simple to implement.
- It is machine independent.
- It works with multiple processes.
- We can have many different critical sections with different semaphores.
- Each critical section has unique access semaphores.
- We can permit multiple processes into the critical section at once, if desirable.

Uses of Semaphores

- For achieving mutual exclusion.
- To solve synchronization problems.

Advantages of Semaphores

- It restricts multiple processes to enter the critical section.
- Semaphores are machine-independent.
- As there is busy waiting in semaphore, there is no wastage of process time and resources.
- It allows flexible management of resources.

Disadvantages of Semaphores

- The operating system has to keep track of all calls to wait and signal semaphore.
- In order to avoid deadlocks in semaphore, the wait and signal operations require to be executed in correct order.
- Semaphore programming is a complicated, so there are chances of not achieving mutual exclusion.
- It is more prone to programming error. It may cause deadlock or violation of mutual exclusion due to programming error.

Monitors

- Monitors are the synchronization construct created to overcome the problems caused by semaphores such as timing errors.
- Monitors are abstract data types and contain shared data variables and procedures.

```
Monitor Monitor_name
{
    data variables;
    Condition variables;
    Procedure P1 {.....}
    Producer P2 {.....}
    Producer Pn {..... }
}
```

Monitors (contd..)

- The shared data variables cannot be directly accessed by a process.
- Procedures are required to allow a single process to access the shared data variables at a time.
- Only one process can be active in a monitor at a time.
- Other processes that need to access the shared variables in a monitor have to line up in a queue and are only provided access when the previous process release the shared variables.

Message Passing

- It is the mechanism for process to communicate and to synchronize their actions.
- In Message passing, processes communicate with each other without resorting to shared variables.
- This method of inter-process communication uses two primitives: **send** and **receive**.
 send (destination, &message);
 receive (source, &message);
- Mode of communication between two processes can take place through two methods: **Direct Communication** and **Indirect Communication**.

Message Passing (contd..)

Direct Communication:

- In this communication, each process that wants to communicate must explicitly name the recipient or sender of the communication.
- The send and receive primitives are defined as:
 - send (P, message) – Send a message to process P.
 - receive (Q, message) – Receive a message from process Q.
- The processes need to know only each other's identity to communicate.
- Direct communication has two addressing schemes: **symmetric scheme** and **asymmetric scheme**.

Message Passing (contd..)

Direct Communication:

- In symmetric scheme, both the sender and the receiver processes must name the other to communicate.
 - send (P, message) – Send a message to process P.
 - receive (Q, message) – Receive a message from process Q.
- In asymmetric scheme, only the sender names the recipient. The recipient is not required to name the sender.
 - send (P, message) – Send a message to process P.
 - receive (id, message) – Receive a message from any process. The variable id is set to the name of process with which communication has taken place.

Message Passing (contd..)

Indirect Communication:

- In this communication, the messages are sent to and received from mailboxes, or ports.
- A mailbox can be viewed abstractly as an object into which messages can be placed by processes and from which messages can be removed. Each mailbox has a unique identification.
- A process communicates with some other process via a number of different mailboxes. Two processes can communicate only if they share a mailbox.
- The send and receive primitives are defined as:
 - send (A, message) – send a message to mailbox A.
 - receive (A, message) – receive message from mailbox A.
- Suppose, the processes P1, P2 and P3 share the mailbox A. Process P1 sends message to mailbox A. This message can be received by process P2 and P3.
- The mailbox can be owned by process or by an operating system.

Bounded Buffer

- A bounded buffer lets multiples producers and multiples consumers share a single buffer. Producers write data to the buffer and consumers read data from the buffer.
- Producers must be blocked if the buffer is full and Consumers must be blocked if the buffer is empty.

Problem 1: The producer wants to put a new data in the buffer, but buffer is already full.

Solution: Producer goes to sleep and to be awakened when the consumer has removed data

Problem 2: The consumer wants to remove data from the buffer but buffer is already empty.

Solution: Consumer goes to sleep until the producer puts some data in buffer and wakes consumer up.

Bounded Buffer (Contd...)

```
#define N 100          /*Number of slots in the buffer*/
int count=0;          /*Number of items in the buffer */
void producer(void)
{
    int item;
    while (TRUE)      //repeat forever
    {
        item=produce_item();    //generate next item
        if(count==N)          //if buffer is full, go to sleep
            sleep();
        inset_item(item);      //put item in buffer
        count=count+1;         //increment count of items in buffer
        if(count==1)           //was buffer empty?
            wakeup(consumer);
    }
}
```

Bounded Buffer (Contd...)

```
void consumer(void)
{
    int item;
    while (TRUE)
    {
        if(count==0)
            sleep();           //if buffer is empty, go to sleep
        item=remove_item();    //take item out of buffer
        count=count-1;        //decrement count of items in buffer
        if(count==N-1)        //check if buffer full?
            wakeup(producer)
        consume_item(item);    //consume item
    }
}
```

Serializability

- When multiple transactions are running concurrently then there is a possibility that the database may be left in an inconsistent state.
- Serializability is a concept that helps us to check which schedules are serializable.
- It always leaves the database in consistent state.
- A serial schedule is always a serializable schedule because in serial schedule, a transaction only starts when the other transaction finished execution. However, a non-serial schedule needs to be checked for serializability.
- A non-serial schedule of n number of transactions is said to be serializable schedule, if it is equivalent to the serial schedule of those n transactions.
- Types of serializability:
 1. Conflict serializability
 2. View serializability

Serializability (contd..)

- **Conflict serializability:** A schedule is called conflict serializable, if we can convert it into a serial schedule after swapping its non-conflicting operations. It can be used to check whether a non-serial schedule is conflict serializable or not.
- **View serializability:** It is a process to find out that a given schedule is view serializable or not. To check whether a given schedule is view serializable, we need to check whether the given schedule is view equivalent to its serial schedule.

T1	T2
R (X)	
W (X)	
	R (X)
	W (X)
R (Y)	
W (Y)	
	R (Y)
	W (Y)



T1	T2
R (X)	
W (X)	
R (Y)	
W (Y)	
	R (X)
	W (X)
	R (Y)
	W (Y)

Locking Protocols

- A lock is a data variable which is associated with a data item. This lock signifies that operations that can be performed on the data items.
- Locks help to synchronize access to the database items by concurrent transactions.
- Transactions processed only once the lock request is granted.
- **Binary Locks:** A binary locks on a data item can be either locked or unlocked state.
- **Shared/exclusive:** This type of locking mechanism separates the locks based on their uses. Different locks are: Shared Lock (S) and Exclusive Lock (X).

Shared Lock (S): It is also called Read-Only Lock. With the shared lock, the data item can be shared between transactions. It will not provide permission to update data item.

Exclusive Lock (X): With this lock, a data item can be read as well as written. This is exclusive and can't be held concurrently on the same data item.

Locking Protocols

- **Simplistic Lock Protocol:** This type of lock-based protocols allow transactions to obtain a lock on every object before beginning operation. Transaction can unlock the data item after finishing the write operation.
- **Pre-claiming Protocol:** It helps to evaluate operations and create a list of required data items which are needed to initiate an execution process. In the situation when all locks are granted, the transaction executes. After that, all locks release when all of its operations are over.
- **Two Phase Locking (2PL) Protocol:** In this protocol, the transaction should acquire a lock after all releases one of its locks. This locking protocol divides the execution phase of a transaction into three different parts:
 - In the first phase, when the transaction begin to executes, it requires permission for the locks it needs.
 - The second part is where the transaction obtains all the locks. When a transaction releases its first lock, the third phase starts.
 - In this third phase, the transaction cannot demand any new locks. Instead, it only releases the acquired locks.

Locking Protocols

- The **Two-Phase Locking protocol** allows each transaction to make a lock or unlock requests in two steps:
 - **Growing Phase:** In this phase, transaction may obtain locks but may not release any locks.
 - **Shrinking Phase:** In this phase, a transaction may release locks but not obtain any new lock.
- The Two-Phase Locking protocol offers serializability. However, it doesn't ensure that deadlocks do not happen.

Time Stamp Protocols

- It uses a timestamp to serialize the execution of concurrent transactions.
- This protocol ensures that every conflicting read and write operations are executed in timestamp order.
- The protocol uses the system time or logical count as a timestamp.
- It gives priority to the older transaction.
- This is the most commonly used concurrency protocol.
- Lock based protocols helps us to manage the order between the conflicting transactions when they will execute. Timestamp based protocols manage conflicts as soon as an operation is created.

Classical IPC Problems

- Dining Philosophers Problems
- The Readers and Writers Problem
- The Sleeping Barber's Problem

Dining Philosophers Problem

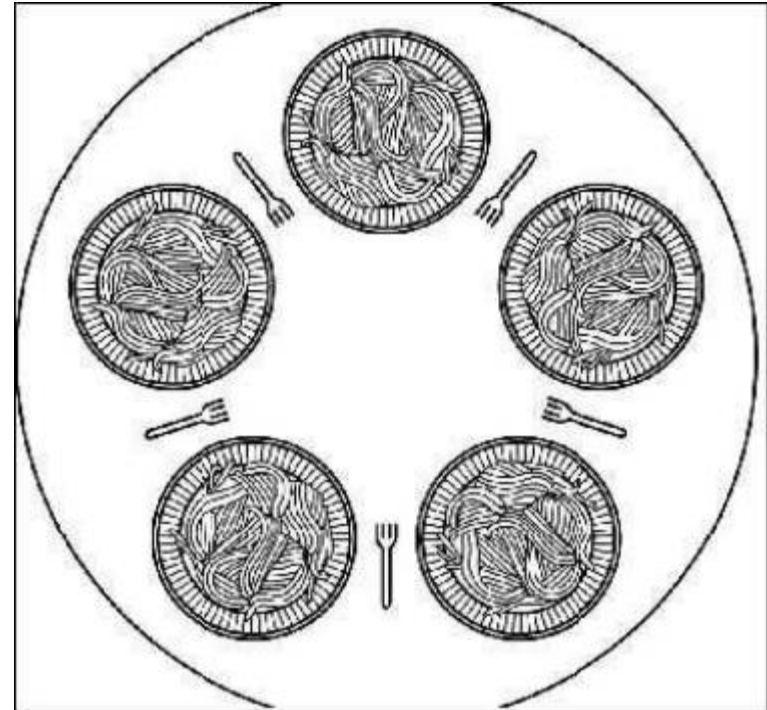
- There are N philosophers sitting around a circular table eating spaghetti and discussing philosophy.
- The problem is that each philosopher needs 2 forks to eat, and there are only N forks, one between 2 philosophers.
- The task is to design an algorithm that the philosophers can follow that insures that none starves as long as each philosophers eventually stops eating and such that the maximum numbers of philosophers can eat at once.
 - Philosophers eat and think alternately
 - Eating needs 2 forks
 - Pick one fork at a time
 - How to prevent deadlock

Classical IPC Problems

The Dining Philosophers Problem

Scenario: consider the five philosophers are seated in a common round table for their lunch, each philosopher has a plate of spaghetti and there is a forks between two plates, a philosopher needs two forks to eat it. They alternates thinking and eating.

What is the solution (program) for each philosopher that does what is supposed to do and never got stuck?



The Dining Philosophers Problem

Solution

Attempt 1: When the philosopher is hungry it picks up a fork and wait for another fork, when get, it eats for a while and put both forks back to the table.

Problem: What happen, if all five philosophers take their left fork simultaneously?

Attempt 2: After taking the fork, checks for right fork. If it is not available, the philosopher puts down the left one, waits for some time, and then repeats the whole process.

Problem: What happen, if all five philosophers take their left fork simultaneously?

Attempt 3: Using semaphore, before starting to acquire a fork he would do a wait() on mutex, after replacing the forks, he would do signal() on mutex.

Problem: adequate but not perfect: only one philosopher can be eating at any instant.

Attempt 4: Using semaphore for each philosopher, a philosopher move only in eating state if neither neighbor is eating. -*perfect solution.*

The Readers and Writers Problem

Statement:

- There is a shared resource which should be accessed by multiple processes. There are two types of processes in this context: reader and writer.
- Any number of readers can read from the shared resource simultaneously, but only one writer can write to the shared resource.
- When a writer is writing data to the resource, no other process can access the resource.
- A writer cannot write to the resource if there is non-zero numbers of readers accessing the resource at that time.

Solution:

- It is evident that readers have higher priority than writer. If a writer wants to write to the resource, it must wait until there are no readers currently accessing that resource.

The Readers and Writers Problem (contd..)

- Here, we use one mutex m and a semaphore w .
- An integer variable `read_count` is used to maintain the number of readers currently accessing the resource.
- The variable `read_count` is initialized to 0.
- A value of 1 is given initially to m and w . Instead of having the process to acquire lock on the shared resource, we use the mutex m to make the process to acquire and release lock whenever it is updating the `read_count` variable.

The Readers and Writers Problem (contd..)

The code for the writer process looks like this:

```
while(TRUE)
{
    wait(w);
    //perform the write operation
    signal(w);
}
```

- The writer just waits on the w semaphore until it gets a chance to write to the resource.
- After performing the write operation, it increments w so that, the next writer can access the resource

The Readers and Writers Problem (contd..)

The code for the reader process looks like this:

```
while(TRUE)
{
    wait(m);          //acquire lock
    read_count++;
    if(read_count==1)
        wait(w);      //release lock
    signal(m);         //perform the reading operation
    wait(m);          //acquire lock
    read_count--;
    if(read_count==0)
        signal(w);     //release lock
    signal(m);
}
```

The Readers and Writers Problem (contd..)

In the code for the reader:

- The lock is acquired whenever the `read_count` is updated by the process.
- When a reader wants to access the resource, first it increments the `read_count` value, then accesses the resource and then decrements the `read_count` value.
- The semaphore `w` is used by the first reader which enters the critical section and the last reader which exits the critical section.
- The reason for this is, when the first reader enters the critical section, the writer is blocked from the resource. Only new readers can access the resource now.
- Similarly, when the last reader exits the critical section, it signals the writer using the `w` semaphore because there are zero readers now and a writer can have the chance to access the resource.

The sleeping Barber Problem

Problem Statement:

There is a barber shop which has one barber, one barber chair, and n chairs for waiting for customers if there are any to sit on the chair.

- If there are no customer, then the barber sleeps in his own chair.
- When a customer arrives, he has to wake up the barber.
- If there are many customers and the barber is cutting a customer's hair, then the remaining customers either wait if there are empty chairs in the waiting room or they leave if no chairs are empty.

Each customer, when he arrives, looks to see what the barber is doing.

- If the barber is sleeping, then the customer wakes him up and sits in the chair.
- If the barber is cutting hair, then the customer goes to the waiting room.
- If there is a free chair in the waiting room, the customer sits in it and waits his turn.
- If there is no free chair, then the customer leaves.

The sleeping Barber Problem (contd..)

Scenario 1:

A customer may arrive and observe that the barber is cutting hair, so he goes to the waiting room. While he is on his way, the barber finishes the haircut he is doing and goes to check the waiting room. Since, there is no one there (the customer not having arrived yet), he goes back to his chair and sleeps. The barber is now waiting for a customer and the customer is waiting for the barber.

Scenario 2:

Two customers may arrive at the same time when there happens to be a single seat in the waiting room. They observe that the barber is cutting hair, go to the waiting room, and both attempt to occupy the single chair.

Solution:

- Using three semaphores.
- First is for the customer which counts the number of customers present in the waiting room (customer in the barber chair is not included, because he is not waiting)

The sleeping Barber Problem (contd..)

- Second, the barber 0 or 1 is used to tell whether the barber is idle or is working.
- Third, mutex is used to provide the mutual exclusion which is required for the process to execute.
- In the solution, the customer has the record of the number of customers waiting in the waiting room if the number of customers is equal to the number of chairs in the waiting room then the upcoming customers leaves the barbershop.
- When the barber shows up in the morning, he executes the procedure barber, causing him to block on the semaphore customers because it is initially 0. Then the barber goes to sleep until the first customer comes up.
- When a customer arrives, he executes customer procedure the customer acquires the mutex for entering the critical region, if another customer enters thereafter, the second one will not be able to until the first one has released the mutex. The customer then checks the chairs in the waiting room if waiting customers are less than the number of chairs then he sits otherwise he leaves and releases the mutex.

The sleeping Barber Problem (contd..)

- If the chair is available then customer sits in the waiting room and increments the variable waiting value and also increases the customer's semaphore this wakes up the barber if he is sleeping. At this point, customer and barber are both awake and the barber is ready to give that person a haircut. When the haircut is over, the customer exits the procedure and if there are no customers in waiting room barber sleeps.